

AI ENTRY POINTS

Версия: 1.0.0

Дата: 2026-06-13

Статус: Active

Автор: Koda (NLP-Core-Team)

НАЗНАЧЕНИЕ

Этот документ определяет порядок чтения документации AI-агентами.

Primary Purpose: Обеспечить консистентный AI workflow при работе с SUMMARY_DOCS.

PRIMARY ENTRY FLOW

Порядок чтения (обязательный):

1. SUMMARY_DOCS/INDEX.md
↓
2. SUMMARY_DOCS/MANIFEST.json
↓
3. SUMMARY_DOCS/summary/ROOT_SUMMARY_DOCS.md
↓
4. SUMMARY_DOCS/modules/MODULE_INDEX.md (для module context)
↓
5. SUMMARY_DOCS/nodes/NODETREE_INDEX.md (для node context)
↓
6. SUMMARY_DOCS/nodes/NODETREE_MANIFEST.json (machine-readable node registry)
↓
7. SUMMARY_DOCS/nodes/NODE_CAPABILITY_MATRIX.md (capabilities by version)
↓
8. SUMMARY_DOCS/nodes/NODE_ACCESS_MATRIX.md (access/security rules)
↓
9. SUMMARY_DOCS/nodes/NODE_DEPENDENCY_MATRIX.md (node dependencies)
↓
10. SUMMARY_DOCS/adr/ADR_INDEX.md (architectural decisions)
↓
11. Relevant node/module contracts (по категории)
↓
12. Relevant runbook (для operational context)
↓
13. Relevant schema (для validation)
↓
14. Relevant health model entry (для monitoring)
↓
15. Relevant ownership metadata (для responsibility)

↓
16. Relevant topology/state (по контексту)

Описание шагов:

Шаг 1: INDEX.md

Цель: Понять общую структуру документации

Что читать:

- Категории документов
- Ключевые документы по разделам
- Статусы документации

Шаг 2: MANIFEST.json

Цель: Получить machine-readable индекс

Что читать:

- Список всех документов
- Metadata (status, audience, tags)
- Related docs links

Шаг 3: ROOT_SUMMARY_DOCS.md

Цель: Понять архитектуру SUMMARY_DOCS

Что читать:

- Принципы работы
- Workflows
- Benefits

Шаг 4: Relevant Contracts

Цель: Получить спецификации для задачи

Что читать (по контексту):

- Node contracts (для работы с узлами)
- Project contracts (для проекта)
- Domain contracts (для доменов)

Шаг 5: Relevant Topology/State

Цель: Получить контекст системы

Что читать (по контексту):

- Network maps (для networking)
- Deployment maps (для deployment)

- State files (для конфигурации)

AI WORKFLOWS

Doc Generation Workflow:

1. Прочитать INDEX.md (overview)
2. Прочитать MANIFEST.json (structure)
3. Проверить наличие дубликатов
4. Создать документ в SUMMARY_DOCS/[category]/
5. Добавить frontmatter
6. Обновить MANIFEST.json
7. Обновить doc-state.json
8. Создать stub в legacy location (если был)

Codegen Workflow:

1. Прочитать INDEX.md (overview)
2. Прочитать MANIFEST.json (structure)
3. Прочитать relevant contracts (specs)
4. Прочитать relevant state (config)
5. Сгенерировать код
6. Проверить соответствие contracts
7. Обновить документацию (если изменилась)
8. Обновить MANIFEST.json
9. Commit changes

Audit Workflow:

1. Прочитать INDEX.md (overview)
2. Прочитать MANIFEST.json (document list)
3. Прочитать каждый документ
4. Проверить актуальность
5. Проверить консистентность
6. Создать audit report
7. Обновить doc-state.json

CONTEXT BUILDING

Minimal Context (для простых задач):

```
{
  "entry_point": "SUMMARY_DOCS/INDEX.md",
  "manifest": "SUMMARY_DOCS/MANIFEST.json",
  "category": "contracts"
}
```

Standard Context (для большинства задач):

```
{
  "entry_point": "SUMMARY_DOCS/INDEX.md",
  "manifest": "SUMMARY_DOCS/MANIFEST.json",
  "overview": "SUMMARY_DOCS/summary/ROOT_SUMMARY_DOCS.md",
  "contracts": ["SUMMARY_DOCS/contracts/node-contracts/"],
  "topology": ["SUMMARY_DOCS/topology/"],
  "state": ["SUMMARY_DOCS/state/"]
}
```

Full Context (для комплексных задач):

```
{
  "entry_point": "SUMMARY_DOCS/INDEX.md",
  "manifest": "SUMMARY_DOCS/MANIFEST.json",
  "overview": "SUMMARY_DOCS/summary/ROOT_SUMMARY_DOCS.md",
  "modules": {
    "index": "SUMMARY_DOCS/modules/MODULE_INDEX.md",
    "manifest": "SUMMARY_DOCS/modules/MODULE_MANIFEST.json",
    "contracts": "SUMMARY_DOCS/modules/contracts/"
  },
  "nodes": {
    "index": "SUMMARY_DOCS/nodes/NODETREE_INDEX.md",
    "manifest": "SUMMARY_DOCS/nodes/NODETREE_MANIFEST.json",
    "contracts": "SUMMARY_DOCS/contracts/nodes/",
    "technical": "SUMMARY_DOCS/nodes/technical/",
    "capabilityMatrix": "SUMMARY_DOCS/nodes/NODE_CAPABILITY_MATRIX.md",
    "accessMatrix": "SUMMARY_DOCS/nodes/NODE_ACCESS_MATRIX.md",
    "dependencyMatrix": "SUMMARY_DOCS/nodes/NODE_DEPENDENCY_MATRIX.md",
    "families": "SUMMARY_DOCS/nodes/NODE_FAMILIES.md",
    "acceptanceChecklist": "SUMMARY_DOCS/nodes/NODE_ACCEPTANCE_CHECKLIST.md"
  },
  "adr": {
    "index": "SUMMARY_DOCS/adr/ADR_INDEX.md",
    "dir": "SUMMARY_DOCS/adr/"
  },
  "policies": [
    "SUMMARY_DOCS/DOC_SOURCE_POLICY.md",
    "SUMMARY_DOCS/DOC_GENERATION_POLICY.md",
    "SUMMARY_DOCS/DOC_CODEGEN_POLICY.md"
  ]
}
```

```

],
"contracts": ["SUMMARY_DOCS/contracts/"],
"summary": ["SUMMARY_DOCS/summary/"],
"topology": ["SUMMARY_DOCS/topology/"],
"state": [
  "SUMMARY_DOCS/state/",
  "SUMMARY_DOCS/nodes/state/"
],
"architecture": ["SUMMARY_DOCS/architecture/"],
"playbooks": [
  "SUMMARY_DOCS/playbooks/",
  "SUMMARY_DOCS/playbooks/node-drift-audit-playbook.md",
  "SUMMARY_DOCS/playbooks/canonical-node-doc-update-playbook.md"
],
"glossary": [
  "SUMMARY_DOCS/appendix/domain-glossary.md",
  "SUMMARY_DOCS/appendix/entity-definitions.md",
  "SUMMARY_DOCS/appendix/entity-relationships.md"
]
}

```

☑ CONFLICT RESOLUTION

При конфликте источников:

Конфликт	Решение
Legacy vs SUMMARY_DOCS	SUMMARY_DOCS wins
Generated vs Canonical	Canonical wins
Old vs New	New wins (check dates)
Contract vs Code	Contract wins

Priority order:

1. **SUMMARY_DOCS canonical** (highest)
2. **Generated mirrors** (read-only)
3. **Compatibility stubs** (redirect)
4. **Legacy documents** (deprecated, lowest)

📄 DOCUMENT CREATION

Frontmatter requirements:

```

---
title: Document Title

```

```
description: Short description
version: 1.0.0
date: 2026-06-13
author: AI Agent Name
status: active|deprecated|generated
audience: human|ai|both
tags:
  - tag1
  - tag2
related_docs:
  - SUMMARY_DOCS/path/to/doc.md
---
```

Content requirements:

- ☒ Clear purpose section
- ☒ Structured headings (H1, H2, H3)
- ☒ Human-readable explanations
- ☒ AI-parsable structure
- ☒ Cross-references к canonical docs
- ☒ No hardware-specific details

VERIFICATION

Перед записью:

```
# 1. Проверить уникальность
grep -r "title: Document Title" SUMMARY_DOCS/

# 2. Проверить категорию
ls SUMMARY_DOCS/[category]/

# 3. Проверить MANIFEST
cat SUMMARY_DOCS/MANIFEST.json
```

После записи:

```
# 1. Валидировать MANIFEST
node scripts/validate-manifest.js

# 2. Обновить doc-state
node scripts/update-doc-state.js

# 3. Проверить ссылки
node scripts/check-links.js
```

BEST PRACTICES

Do:

- ☒ Всегда читать INDEX.md первым
- ☒ Проверять MANIFEST.json на дубликаты
- ☒ Использовать canonical paths
- ☒ Обновлять MANIFEST.json при изменениях
- ☒ Следовать policies

Don't:

- ☒ Не создавать документы вне SUMMARY_DOCS
- ☒ Не использовать legacy paths
- ☒ Не дублировать canonical документы
- ☒ Не игнорировать policies
- ☒ Не забывать обновлять MANIFEST.json

RELATED DOCUMENTS

- [INDEX.md](#) — Главная навигация
- [MANIFEST.json](#) — Индекс документов
- [ROOT_SUMMARY_DOCS.md](#) — Обзор SUMMARY_DOCS
- [DOC_GENERATION_POLICY.md](#) — Политика генерации
- [DOC_CODEGEN_POLICY.md](#) — Политика кодогенерации
- [codegen-playbook.md](#) — Codegen инструкции

Создано: 2026-06-13

Версия: 1.0.0

Статус: Active

Автор: Koda (NLP-Core-Team)

 **Baloo - Проверни общение!**